Smart Office Edge AI: Refined Staged Prompts + README Updates

How to use: paste the SHARED PREAMBLE + one stage at a time into your AI assistant. Save the output, run it, then move to the next stage. Never ask for the whole app in one prompt.

PART A: STAGED PROMPTS

SHARED PREAMBLE (paste at the top of every stage)

```
ROLE
Act as a Senior Computer Vision & Edge AI Engineer building "Smart Office
Edge AI
Workstation Assistant" as ONE single-file Python script, app.py. 100% local,
Windows
10/11, CPU + integrated webcam. No cloud, no paid APIs, no training, no GPU,
no
network requests.

PRODUCT CONTEXT (user stories, these define what "done" means)
US1 - Fast lock when I leave. "As an enterprise employee handling sensitive
data, I want
  my workstation to detect when I step away and lock after a configurable
absence
  threshold, so passersby cannot read confidential work, without false locks
while I am
  still present."
  Positioning: this is FAST LOCK ON LEAVING, not a replacement for the
Windows idle
  timer. Windows locks only after inactivity (and can lock a person who is
reading);
  this app locks quickly when no face/hand evidence is present and never
locks while
  evidence is present. Do NOT claim power saving. Instead show measured
CPU/RAM so the
  cost is visible and honest.
  Acceptance: brief dropout never locks; lock fires once per absence episode
after
  ABSENCE_TIMEOUT; no lock when camera is covered/lost/dark.
US2 - Touchless presentation and media control. "As a presenter or lab
researcher with
  occupied hands, I want to advance slides and toggle media playback with
hand gestures
  seen by my webcam, so I can interact without touching input devices."
  Acceptance: open palm = space, point right = right, point left = left; each
sustained
  gesture fires once; keys go only to an armed, non-sensitive target window.
US3 - Privacy assurance and air-gapped compliance. "As a privacy-focused user
and IT
  auditor, I need all visual processing to run in transient RAM on the local
CPU, with no
```

```
  frames, faces or video written to disk and no data sent over the network."
  Acceptance: no image/video files created; no sockets opened; no face
identification;
  SQLite holds only settings, minimal events, session metrics; window titles
are never
  logged.

VIBE-CODING RULES (non-negotiable)
R1 Single file, clearly commented sections, small pure functions, injected
clock.
R2 Never train. Pre-trained MediaPipe only (face_detection + hands). No
YOLO/datasets.
R3 Hard debounce: hold time + ACTION_COOLDOWN + neutral-state re-arm; one
sustained
   gesture fires at most once.
R4 Safety-first: dry-run is DEFAULT; real actions need --live. 15 s timeout
in dev,
   --demo = 5 s.
R5 Native Windows lock via ctypes.windll.user32.LockWorkStation(); no extra
   system-management dependencies.
R6 Fail safe, but visibly: unreliable data never causes an action and always
shows a
   warning.
R7 Honest reporting: you cannot run code or pip. Label every check "not
executed" /
   "requires manual test". Dependency compatibility is "from knowledge,
verify with
   pip check". Never invent FPS, latency, or test results.

STATE PRIORITY (highest first; evaluate in this order every frame)
1. CAMERA_LOST  2. SENSOR_UNRELIABLE  3. LOCKED  4. ABSENT  5. GRACE  6.
PRESENT
- Higher-priority conditions override lower ones.
- Camera recovery must NOT reset to PRESENT if the state was LOCKED: it
returns to
  LOCKED and leaves it only after RETURN_CONFIRM_SECONDS of continuous person
evidence.
- CAMERA_LOST and SENSOR_UNRELIABLE reset the absence timer and never lock.

CONFIG (named constants at top; validate ALL values and CLI args before
opening camera)
ABSENCE_TIMEOUT=15.0 (3-60; --demo=5.0), GRACE_PERIOD=1.0,
ACTION_COOLDOWN=1.5 (0.5-3.0),
GESTURE_HOLD_TIME=0.4, POINT_DX_THRESHOLD=0.08, LOW_LIGHT_THRESHOLD=40,
BLOCKED_LENS_STD_THRESHOLD=8, CAMERA_FAIL_FRAMES=30,
```

```
CAMERA_RETRY_SECONDS=2.0,
CAMERA_MAX_RETRIES=30, RETURN_CONFIRM_SECONDS=1.0, TARGET_ARM_COUNTDOWN=3.0,
LOG_RETENTION_DAYS=7, DB_PATH="smart_office.db",
UNRELIABLE_LOCK_SECONDS=None (optional, see Stage 3),
GESTURE_ACTIONS=
{"OPEN_PALM":"space","POINT_RIGHT":"right","POINT_LEFT":"left"},
SENSITIVE_TITLE_KEYWORDS=("password","sign in","log
in","login","credential","bank",
"payment","windows security","user account
control","passkey","2fa","verification"),
plus named LANDMARK_MIN_VISIBILITY / consistency-frame constants.

STACK
Python 3.11, opencv-python==4.10.0.84, mediapipe==0.10.14 (legacy solutions
API),
numpy==1.26.4, pyautogui, psutil (optional, try/except). State pins as
unverified.
face_detection(model_selection=0, min_detection_confidence=0.6);
hands(max_num_hands=1, min_detection_confidence=0.7,
min_tracking_confidence=0.6).

ANSWER FORMAT: start with at most 10 lines listing contradictions you found
and how you
resolved them (these instructions win). Then deliver only what the stage asks
for.
```

STAGE 1: Pure logic, config, database and self-tests (NO camera, NO GUI)

```
[PASTE SHARED PREAMBLE FIRST]

STAGE 1 OF 3. Write app.py containing ONLY sections CONFIG, VALIDATION,
DATABASE,
OS ACTIONS (stubs), GESTURES (pure classification), STATE MACHINE, SELF-
TESTS, and a
MAIN that supports only --selftest and --report. Do not open the camera or
import
mediapipe in this stage.

1. VALIDATION: reject bad config/CLI values with a clear message.
2. STATE MACHINE: a pure class/function step(now, face_count, hand_seen,
image_reliable,
   camera_ok) -> (state, actions). Implement the STATE PRIORITY order above.
   - Lock at most once per absence episode; LOCKED exits only after
     RETURN_CONFIRM_SECONDS of continuous evidence.
   - Camera recovery respects LOCKED (never wipes it).
   - If UNRELIABLE_LOCK_SECONDS is set, a sensor that stays unreliable longer
than that
     value may lock only when OS input idle time (injected function,
GetLastInputInfo
     in real use) also exceeds the same value. Default None = disabled.
3. LOCK RESULT: the lock executor returns bool (the LockWorkStation return
value). The
   state machine records "LOCK REQUESTED" (accepted) or "LOCK FAILED"
(rejected). Never
   say "EXECUTED"/"LOCKED OK": the OS call only confirms the request was
accepted.
   Simulation says "SIMULATED: WOULD LOCK".
4. GESTURES: pure function classify(landmarks) -> OPEN_PALM / POINT_RIGHT /
POINT_LEFT /
   NONE with the geometric rules (palm: 4 fingertips above PIP and thumb_tip
farther from
   pinky_mcp than thumb_ip; point: index extended, others curled, |dx|>|dy|
and
   |dx|>POINT_DX_THRESHOLD). Include a landmark-in-bounds check. Plus a pure
debouncer
   (hold, cooldown, neutral re-arm) with injected clock.
5. TARGET GATE as a pure function allow_key(armed_hwnd, fg_hwnd, fg_title,
locked,
   gestures_on). Rules: fail CLOSED if title is empty/None/unreadable; block
on any
   SENSITIVE_TITLE_KEYWORDS match (case-insensitive); block if handle
```

```
differs; block when
   LOCKED. Return (allowed, reason). Caller must DISARM the target on a
handle mismatch.
   Window titles are never logged or stored.
6. DATABASE (sqlite3, parameterized): events, settings, sessions tables as in
the
   README; log state CHANGES only; event types: APP_START, APP_STOP,
PRESENT_TO_AWAY,
   AWAY_TO_PRESENT, AUTO_LOCK (detail: simulated / requested / failed),
GESTURE_ACTION,
   CAMERA_LOST, CAMERA_RESTORED, SENSOR_UNRELIABLE, CAMERA_GAVE_UP,
FAILSAFE_TRIGGERED,
   LIVE_MODE_DISABLED_ON_ERROR. Retention cleanup at startup. Persist and
validate
   ABSENCE_TIMEOUT, ACTION_COOLDOWN, gestures_enabled, auto_lock_enabled;
NEVER persist
   live mode. Corrupt DB -> rename to .corrupt and start fresh.
7. --report: last 20 events; locks split simulated vs requested; gestures per
type split
   the same way; avg FPS/latency from sessions ("no sessions stored" if
empty).
8. SELF-TESTS (unittest, fake clock, mocked executors, temp DB): brief
dropout no lock;
   lock only after timeout; once per episode; LOCKED exit needs
RETURN_CONFIRM_SECONDS;
   priority order (CAMERA_LOST beats SENSOR_UNRELIABLE beats LOCKED); camera
recovery
   while LOCKED stays LOCKED; unreliable/camera-lost never lock and reset
timer;
   UNRELIABLE_LOCK_SECONDS rule; lock-executor False -> "LOCK FAILED"; each
gesture fires
   once until cooldown + neutral; simulation never calls executors; target
gate blocks
   sensitive/foreign/empty-title windows; validation rejects bad values;
settings
   round-trip + bad-value fallback; retention; report maths. Print PASS/FAIL
per test.

OUTPUT: contradictions (max 10 lines), install commands (unverified), the
full app.py in
ONE code block, run commands for --selftest and --report, and a table of the
tests
written with status "not executed".
```

Your check before Stage 2: run `python app.py --selftest` and note real PASS/FAIL in the checklist. Fix failures first (paste the traceback into the assistant).

STAGE 2: Camera, detection, HUD, live actions

```
[PASTE SHARED PREAMBLE FIRST]

STAGE 2 OF 3. Below is my working Stage 1 app.py (all self-tests pass: <PASTE
RESULT>).
Extend it, keeping every Stage 1 function and test intact. Add DETECTION, OS
ACTIONS
(real), HUD and the full MAIN. Return the complete updated app.py.

<PASTE app.py HERE>

ADD:
1. Camera: index 0 or --camera N, 640x480, cv2.flip(frame,1) BEFORE
processing, BGR->RGB,
   frames only in RAM, nothing written to disk. Handle open failure, mid-run
disconnect,
   bounded retries then persistent "camera unavailable - restart app" (never
locks).
   Main loop in try/finally: release camera, destroy windows, write sessions
row, close
   DB.
2. Image reliability: mean gray < LOW_LIGHT_THRESHOLD or std-dev <
   BLOCKED_LENS_STD_THRESHOLD -> image_reliable=False. Feed the existing
state machine.
3. Real OS actions: LockWorkStation via ctypes (check and use the return
value; HUD and
   log show "LOCK REQUESTED" or "LOCK FAILED", never "EXECUTED: LOCK"). Keys
via pyautogui
   with FAILSAFE=True, PAUSE=0, only space/left/right. FailSafeException ->
simulation
   mode, red HUD warning, log, keep running. macOS/Linux branches labelled
"untested, not
   supported".
4. Target gate in live mode: 't' starts a 3 s countdown, then records the
foreground
   handle (ignore this app's own window). On handle mismatch DISARM and show
"FOREGROUND
   CHANGED". Unreadable/empty title -> "UNREADABLE TITLE" (blocked). Titles
are never
   logged.
5. Catch-all error handler around the per-frame processing and the action
executors
   (except Exception): disable live mode, cancel pending gestures, disarm
target, log
```

```
   LIVE_MODE_DISABLED_ON_ERROR if the DB is available (never crash on a DB
error), show
   a red "ERROR - LIVE DISABLED" warning, and keep the loop alive in
simulation mode.
6. HUD (OpenCV only): dark top banner with rolling FPS, average latency ms,
state,
   people count, gesture and the action it WOULD perform, hold-progress bar,
last action
   (SIMULATED / REQUESTED / FAILED), absence countdown, CPU/RAM if psutil
exists.
   Persistent "CAMERA ACTIVE - local processing only" and mode badge (green
SIMULATION,
   red LIVE AUTOMATION ENABLED). Warning banner for SENSOR_UNRELIABLE /
CAMERA_LOST /
   ERROR. State text must say FACE/HAND evidence, not "person detected".
Trackbars for
   timeout (3-60) and cooldown (x10 integer scale, 0.5-3.0). Keys: q/Esc
quit, l auto-lock,
   g gestures, d sim/live (only if --live), t arm target. One-time console
note if FPS
   stays below 15. No per-frame console output.
7. Do not add any network call, image write, or face identification.

OUTPUT: contradictions (max 10 lines), full updated app.py in ONE code block,
run commands
(default, --demo, --live, --live --demo, --report, --selftest), known
limitations (max
8), and the testing checklist (12 scenarios) with status "not executed".
```

STAGE 3: Bug fixes from real testing

```
[PASTE SHARED PREAMBLE FIRST]

STAGE 3 OF 3. Here is my current app.py and the REAL problems I observed when
running it
(copy actual error text and behaviour; do not paraphrase):

<PASTE app.py>

OBSERVED PROBLEMS:
1. <e.g. Left/Right reversed>   2. <traceback>   3. <HUD flicker>   ...

Fix ONLY these problems. Rules: smallest possible change, do not rewrite
untouched
sections, do not change the config defaults, safety rules or state priority
order, and
keep all self-tests passing (add one regression test per bug where it is
logic). Return
(a) a bullet list of root cause + fix per problem, and (b) the complete
updated app.py.
State clearly which fixes you could not verify (you cannot run code).
```

PART B: README UPDATES (replace or add these sections)

Section 1 (replace the User Stories / overview intro)

Problem and value. Windows already locks after inactivity, but that timer cannot tell "reading quietly at the desk" from "walked away," so it must be long. This app provides **fast lock when you leave, with no false lock while you are present**, plus touchless presentation control, entirely on-device.

User story	Value	How it is verified
US1 Fast lock on leaving	Short, configurable absence threshold; no lock while face/hand evidence exists	`--selftest` state tests, manual walk-away test
US2 Touchless slides/media	Hands-free slide and play/pause control with debounce and target gate	Gesture tests, manual slide test
US3 Privacy and air-gapped	Frames processed in RAM only; no disk images, no network	Manual file check and firewall/Wireshark

OS timer vs presence lock (add to README and talking points):

	Windows inactivity timer	This app
Trigger	No keyboard/mouse input for N minutes	No face/hand evidence for ABSENCE_TIMEOUT seconds
Typical delay	Minutes	5-60 s (configurable)
False lock while reading	Yes, if no input	No, while a face is visible
Fails if	Never (OS level)	Poster/bystander face visible; lens covered (fail-open)
Role	Primary protection	Convenience layer on top; keep OS lock enabled

Section 2 (replace the power-saving claim)

Remove "energy saving" from the overview, talking points and impact statements. Replace with: *"Reduces the exposure window after you leave. The HUD shows measured CPU/RAM; a running webcam plus MediaPipe may use more power than idle sleep, so no power saving is claimed."* Record measured CPU/RAM in the evidence table below.

Section 3 (add to Known limitations; cap raised to 10)

- Face/hand evidence is not person detection: facing away, photos or posters can mislead it.
- **Any face keeps the PC unlocked:** a bystander or a poster in view counts as presence.
- **Covered lens fails open:** SENSOR_UNRELIABLE suspends locking by design, so covering the lens disables auto-lock. Optional mitigation: set `UNRELIABLE_LOCK_SECONDS` (locks only if OS input idle time also exceeds it).
- Sensitive to lighting; very dim or backlit scenes cause dropouts.
- Thumb and hand-orientation heuristics are fragile; angles other than a frontal palm may fail.
- Partial hand visibility (hand at frame edge) is rejected, so gestures there are ignored.
- Mirroring assumes a standard front-facing webcam view.
- Target-window guard is best-effort and cannot judge what the application is showing; it fails closed on unreadable titles and disarms on window change.
- Windows is the only supported platform; macOS/Linux lock commands are untested.
- `LockWorkStation()` success means the request was accepted, not that the screen is verified locked (HUD shows "LOCK REQUESTED").
- The app depends on Windows' own security; it does not replace OS inactivity-lock policy.

Section 4 (replace the conflict-resolution table additions: rows 13-17)

#	Flaw	Resolution
13	Overlap with OS idle lock	Reframed as fast-lock-on-leave; comparison table added
14	Unsupported power-saving claim	Removed; CPU/RAM measured and reported
15	Ambiguous overlapping states	Priority: CAMERA_LOST > SENSOR_UNRELIABLE > LOCKED > ABSENT > GRACE > PRESENT; recovery respects LOCKED
16	"EXECUTED: LOCK" overclaim	Return value checked; "LOCK REQUESTED" / "LOCK FAILED"
17	Narrow error handling and target-gate gaps	Catch-all disables live mode; disarm on mismatch; fail closed on unreadable titles; titles never logged

Section 5 (replace the "one prompt in one hour" runbook)

Use the three staged prompts above: **Stage 1** (logic + self-tests, ~15 min) -> run `--selftest` -> **Stage 2** (camera + HUD, ~20 min) -> manual tests -> **Stage 3** (bug fixes, ~15 min) -> fill the evidence tables. If time runs out, Stage 1 + `--selftest` alone is still a valid, honest deliverable.

Section 6 (Evidence tables: fill with REAL results only)

Test checklist: keep the 12-row table but change every Status cell to `PASS` / `FAIL` / `not executed` as you actually run it, with the date and a one-line note.

Measured performance (fill from the HUD, 60 s per row):

Scenario	FPS	Latency (ms)	CPU %	RAM (MB)	Notes
Idle, face visible					
Gesture active					
Absent (countdown)					
Dim room					

Section 7 (talking-points tweak)

Replace "real-world impact (workstation security, energy saving, accessibility)" with "workstation security (fast lock on leaving), accessibility, and privacy by design." Add a 30-second slide: "OS timer vs presence lock, and where our approach fails (poster, covered lens)."

Section 8 (reflection log seed rows, from the flaw review)

#	What the AI got wrong	How we fixed it
1	Overclaimed power saving	Removed claim, measured CPU/RAM
2	"EXECUTED: LOCK" without checking the OS result	Checked return value, HUD says LOCK REQUESTED
3	Camera recovery could overwrite LOCKED	Added state priority order and a regression test
4	Target gate passed empty titles	Fail closed on unreadable titles
5	(add what you really find)	